

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 2

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : Design and implement a Multilayer Perceptron (MLP) for classification of the Iris or Wine dataset, and evaluate its performance using accuracy and a confusion matrix.

Algorithm :

1. Load the Iris dataset.
2. Split the dataset into training (80%) and testing (20%) sets.
3. Standardize the training and testing features using StandardScaler.
4. Initialize the MLP classifier with hidden layers (10,10), ReLU activation, Adam optimizer, and max_iter=1000.
5. Train the MLP model using the training data.
6. Predict the class labels for the test data.
7. Evaluate the model using Accuracy, Confusion Matrix, and Classification Report.
8. Visualize the Confusion Matrix and performance metrics.
9. Stop.

Code :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import pandas as pd

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

mlp = MLPClassifier(
    hidden_layer_sizes=(10,10),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

report = classification_report(y_test, y_pred)
print("\nClassification Report:\n", report)
```

```

#visualization of confusion matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Class 0", "Class 1", "Class 2"]
)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix")
plt.show()

report_dict = classification_report(
    y_test,
    y_pred,
    target_names=["Class 0", "Class 1", "Class 2"],
    output_dict=True
)

df_report = pd.DataFrame(report_dict).transpose()

plt.figure(figsize=(8,5))
plt.imshow(df_report.iloc[:-1, :-1], cmap="YlGnBu", aspect="auto")

plt.xticks(range(len(df_report.columns[:-1])), df_report.columns[:-1])
plt.yticks(range(len(df_report.index[:-1])), df_report.index[:-1])

for i in range(len(df_report.index[:-1])):
    for j in range(len(df_report.columns[:-1])):
        plt.text(j,
            i,
            f"{df_report.iloc[i, j]:.2f}",
            ha="center",
            va="center",
            color="black"
        )

plt.colorbar(label="Score")
plt.title("Classification Report")
plt.xlabel("Metrics")
plt.ylabel("Classes")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7,5))

```

```
plt.plot(mlp.loss_curve_, marker='o')
plt.title("MLP Training Loss Curve")
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

metrics = ['Precision', 'Recall', 'F1-score']

macro = [0.97, 0.96, 0.97]
weighted = [0.97, 0.97, 0.97]

x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(7,5))
plt.bar(x-width/2, macro, width, label='Macro Avg')
plt.bar(x+width/2, weighted, width, label='Weighted Avg')

plt.xticks(x, metrics)
plt.ylim(0.9,1.0)
plt.ylabel("Score")
plt.title("Macro Average vs Weighted Average")
plt.legend()
plt.grid(axis='y')
plt.show()
```

Output :

```
... Accuracy: 0.9666666666666667
```

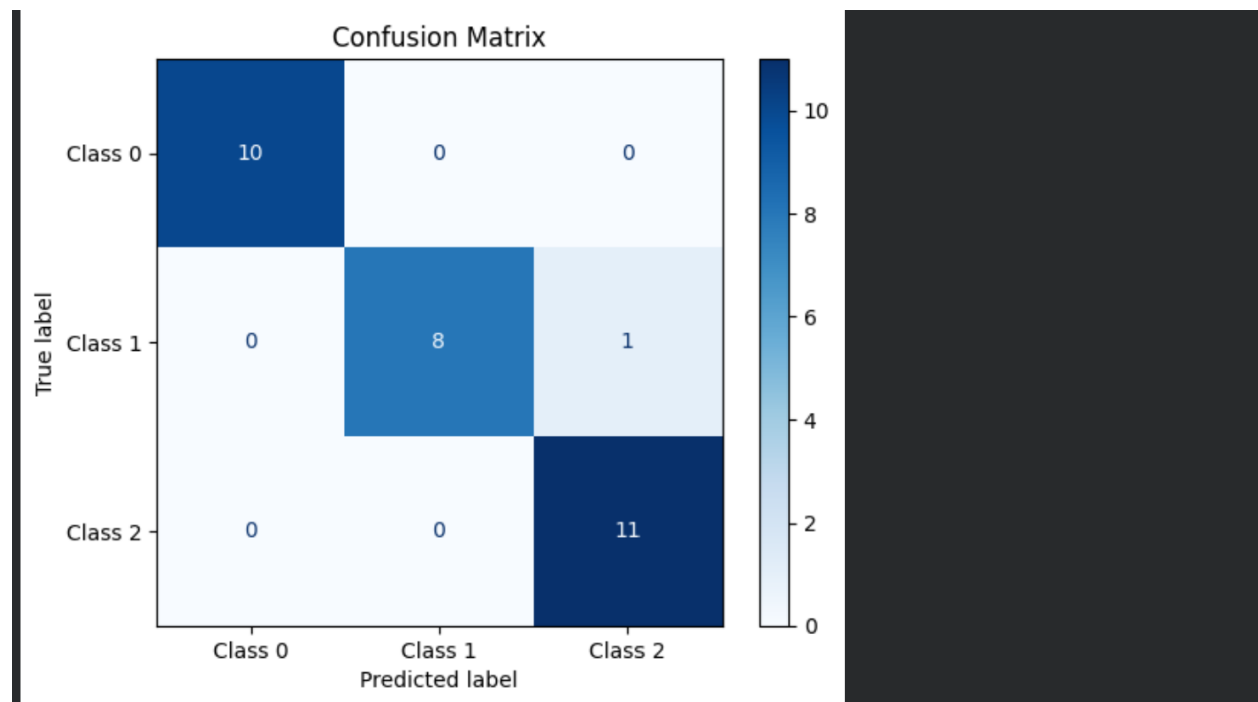
```
Confusion Matrix:
```

```
[[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

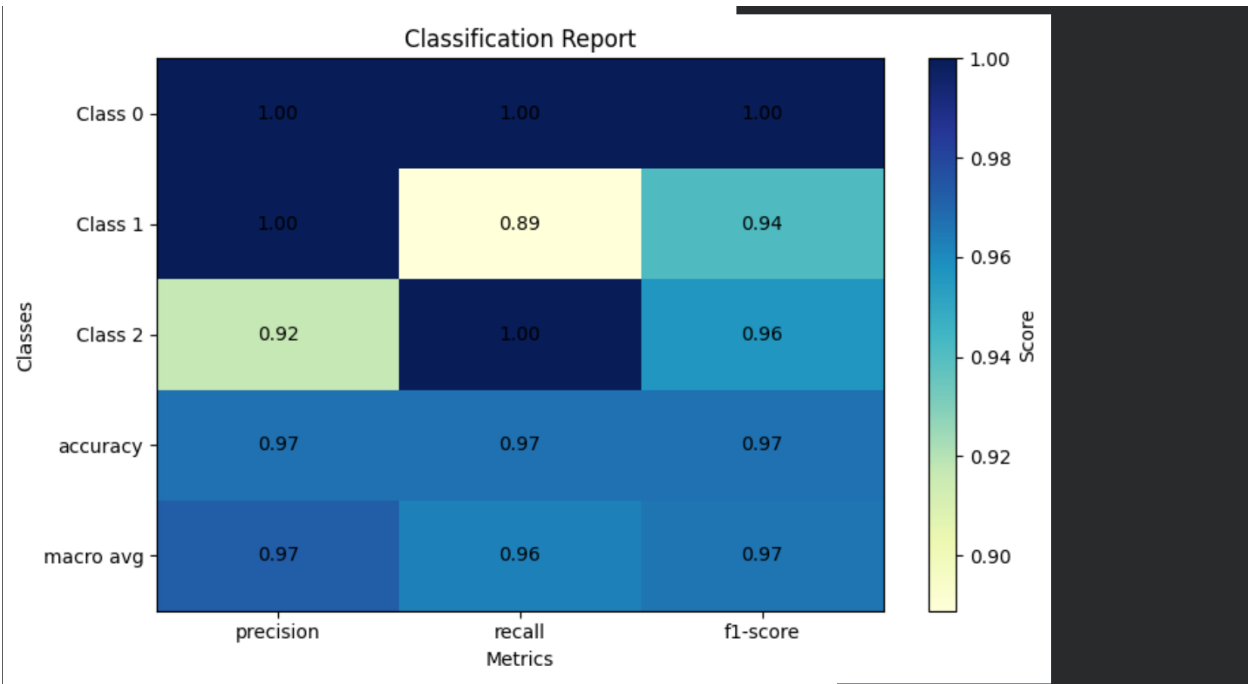
```
Classification Report:
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	0.89	0.94	9
2	0.92	1.00	0.96	11
accuracy			0.97	30
macro avg	0.97	0.96	0.97	30
weighted avg	0.97	0.97	0.97	30

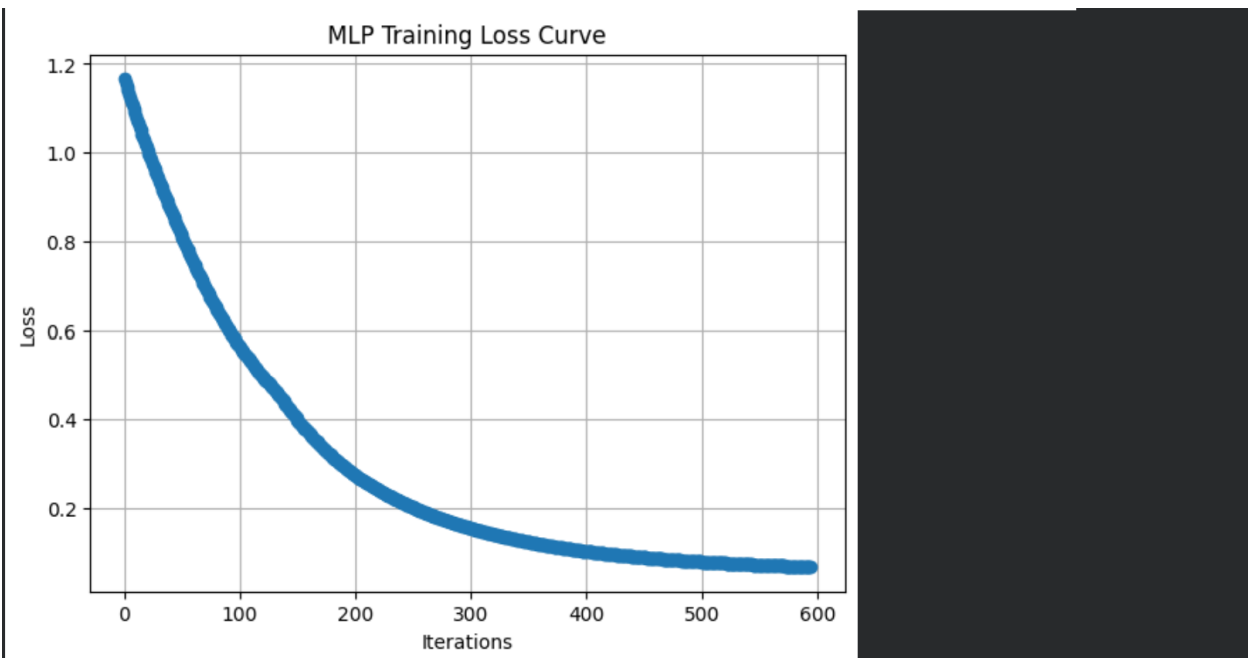
- Confusion matrix visualization:



- **Classification Report visualization**



- **Training Loss Curve Visualization**



- **Macro Average vs Weighted Average visualization**

